

EXAM OPERATING SYSTEMS

1. Scrieti o comanda UNIX care afiseaza toate liniile din fisierul a.txt care contin cel putin un numar binar multiplu de 4 cu cinci sau mai multe cifre (ex: 010100).

Answer: `grep -E "[0,1]{3,}[0]{2}" a.txt`

For a binary number that is a multiple of 4, we cannot have 2^0 and 2^1 (we start from 2^2 ...).

2. Scrieti o comanda UNIX care inverseaza toate perechile de cifra impara urmata de vocala.

Answer: `sed -E "s/([13579])([aeiou])/^2\1/gi"`

We need parantheses to mark the groups in the regex. We use the flags g(for replacing everywhere, not only on the first occurence) and i(to make it case-insensitive).

3. Scrieti o comanda UNIX care afiseaza toate scorurile de fotbal unice (ex: 4-0) care apar in fisierul a.txt. Numarul de goluri poate avea maximum doua cifre

Answer: `grep -E -o "[0-9]{1,2}-[0-9]{1,2}" a.txt | sort -u`

We use -o to only see the scores(the matched part), not the whole line.

`sed -En 's/.*\b([0-9]{1,2}-[0-9]{1,2})\b.*^1/p' a.txt | sort -u`

This works as well, but the “b” part is a bit tricky (might not work all the time).

4. Afisati numarul de procese ale fiecarui utilizator activ din sistem

Answer: `ps -eo user= | sort |uniq -c`

Ps – shows all processes (-e for the running ones)

-o for output format

User= (shows the user name, but removes the header)

5. Scrieti un script Shell UNIX care calculeaza media de fisiere cu extensia .txt per director din directorul curent si toate subdirectoarele lui.

```
#!/bin/bash
```

```
find . -type d | while read dir; do
    files = $(find $dir -type f -name "*.txt")

    count=0
    sum=0

    for f in $files
    do
        lines=$(wc -l < $f)
        sum=$((sum+lines))
        count=$((count+1))
    done

    if [ $count -ne 0 ]
    then
        echo "$($sum/$count)"
    fi
done
```

Something like this?? I guess??

6. Cate procese va crea fragmentul de cod de mai jos, excluzand procesul parinte initial ?

```
if(fork()!=fork()){
    fork();
}
```

Answer: 6 processes (without the parent process)

Explicatie Gemini :)))

Atunci când evaluezi o condiție complexă precum `if (fork() != fork())`, secretul este să reții că **fiecare apel `fork()` se execută pe rând**, iar fiecare apel dublează numărul de procese existente în acel moment.

Iată cum se ajunge la cele **6 procese noi**:

1. **Primul `fork()`**: Procesul inițial se dublează. Avem acum **2 procese**.
2. **Al doilea `fork()`**: Ambele procese existente se dublează din nou. Ajungem la **4 procese**.
3. **Evaluarea condiției**: Fiecare dintre cele 4 procese verifică rezultatele celor două apeluri (`valoare_stanga != valoare_dreapta`).
 - Pentru 3 dintre ele, valorile sunt diferite (Adevărat), așa că intră în `if`.

- Pentru 1 singur proces (ultimul copil creat), valorile sunt egale (\$0 == 0\$, adică Fals), așa că acesta sare peste if.
4. **Al treilea fork() (în if):** Cele 3 procese care au intrat în if se dublează și ele, generând încă **3 procese noi**.

Adunând toate procesele nou create în fiecare etapă (\$1 + 2 + 3\$), obținem un total de **6 procese noi** (sau 7 procese în total, dacă îl numărăm și pe cel părinte de la care s-a pornit).

7. Ce tipareste in consola fragmentul de cod de mai jos ?

```
char* s[3] = {"A", "B", "C"};
for(i=0; i<3; i++)
{
    if(fork() != 0) {
        execl("/bin/echo", "/bin/echo", s[i], NULL);
    }
}
```

Answer: “ABC” (on different lines; and maybe in a random order? It depends, but ideally in this order). The if statement is in the parent process and the instructions after exec(if it’s indeed successful) do not get executed. Thus we continue with the child, which, in its turn, gets forked.

SIDENOTE: because the parent finishes after that exec call, the children are “adopted” by the unix system itself, which will free them at the end from the zombie state(it’s like we would have called wait(0) in the main function).

8. Ce face apelul sistem “write” cand in PIPE este spatiu, dar nu suficient pentru cat i se cere sa scrie?

Answer: the write function might not write all the data that was given, but it will definitely return the number of bytes(the length) that it has written

9. Ce tipareste fragmentul de cod de mai jos daca niciun alt proces nu deschide FIFO-ul “abc”? Justificati raspunsul.

```
int w, n, k=10, r;
r = open("abc", O_WRONLY);
n = write(r, &k, sizeof(int));
printf("%d\n", n);
```

The open function gets bugged because no reader is open. So the program gets stucked and we do not have any output (it generates a SIGPIPE error signal).

10. Ce se intampla cu procesele zombie ale caror parinte s-au terminat?

Answer: Zombie processes remain in the process table after termination, keeping their PID and exit status. If their parent process terminates, they are adopted by init/systemd and are eventually cleaned up. If too many zombies accumulate, they can exhaust the process table, preventing new processes from being created.

11. Considerati ca functia f este executata simultan de 10 thread-uri. Adaugati liniile de cod necesare ca sa asigurati ca n va avea valoarea 10 dupa ce thread-urile isi incheie executia ?

```
int n=0;
pthread_mutex_t mtx=PTHREAD_MUTEX_INITIALIZER;

void* f(void* p) {
    pthread_mutex_lock(&mtx);
    n++;
    pthread_mutex_unlock(&mtx);
    return NULL;
}

int main()
{
    pthread_t th[10];
    for (int i=0;i<10;i++)
    {
        pthread_create(&th[i], NULL, f, NULL);
    }
    for (int i=0;i<10;i++)
    {
        pthread_join(th[i],NULL);
    }
    pthread_mutex_destroy(&mtx);
    return 0;
}
```

12. Planificati executia job-urilor urmatoare (date ca Nume/Durata/Termen) incat suma intarzierilor job-urilor sa fie minima: A/22/27, B/2/15, C/4/5

First, sort by duration: C(5), B(15), A(27)

C starts at 0 -> finishes at 4

B starts at 4 -> finishes at 6

A starts at 6 -> finishes at 28

So we have a delay of 1

⇒ Order is C, B, A

13. Ce se intampla cu un link hard cand fisierul spre care puncteaza este sters? (fisier = inode care arata catre un bloc de date)

For a hard link, the i-node points towards the data, so when we delete a file, the data remains.

14. Write a UNIX Shell command that displays the lines in a file a.txt that contains words starting with capital letters.

Answer: `grep -E "[A-Z][A-Za-z]*" a.txt`

15. Write a UNIX Shell command that inverts in file a.txt all pairs of neighboring digits

Answer: `sed -E "s/([0-9])([0-9])\1/g" a.txt`

16. File a.txt contains on each line two numbers separated by space. Write a UNIX Shell command that displays for each line the sum of its numbers.

Answer: `awk '{print s=$1 + $2}' a.txt`

Or `awk '{sum = $1 + $2; print sum}' a.txt`

17. Display only the lines of file a.txt that appear only once (not duplicated)

Answer: `cat a.txt | sort | uniq`

18. Write a UNIX Shell script that displays the name of each .txt file in the current directory that contains the word "cat"

```
find . -type f -name "*.txt" | while read filename
do
    if grep -E -q "\bcat\b" $filename
    then
        echo $filename
    fi
done
```

19. What does the system call "read" do when the pipe is empty.

Answer: when the pipe is empty the read call waits for data to arrive in the pipe or for no writer to be connected to the pipe.

20. What does the system call “open” do before returning from opening a FIFO?

Answer: it waits for the complementary operation to be performed. When we open it to read, we wait that someone opens for writing; when we open to write, we wait for someone to read.

21. Give a reason for choosing threads over processes

Answer: Threads are chosen over processes because they share the same memory space, making communication and data sharing faster and easier. They also have lower overhead than processes, since creating and switching threads is more efficient than creating and switching processes.

22. What is a binary semaphore and what is the effect of its P method, when called by multiple concurrent processes/threads?

Answer: A binary semaphore is a synchronization method that can take only two values, 0 and 1, and is used to provide mutual exclusion like a mutex. The P (wait) operation decrements the semaphore; if its value is 0, the calling thread/process blocks until another thread performs a V (post) operation, which increments the semaphore and wakes up one waiting thread.

23. Give three regular expressions that match any non-negative number multiple of 5.

Answer: `[0-9]*[05]`

`(0|[1-9][0-9]*)[05]`

`(^[^0-9])(0|[1-9][0-9]*)[05]([^\0-9]|$)`

24. Give five GREP commands that display all the lines in a file that contain letter "a" uppercase or lowercase.

Answer: `grep -E -i "a" a.txt`

`grep -E "[aA]" a.txt`

`grep -E -i "^.*a.*$" a.txt`

`grep -E "^.*[aA].*$" a.txt`

`grep -E -i "A" a.txt`

25. Write two SED commands that display from a file only the lines that do not contain digit 7.

Answer: sed -E "s/([^\n])^1/g"

sed -E "s/(.*)([^\n])(.*)^1\2\3/g"

26. Write an AWK command that displays the sum of the next to last field of all lines.

Answer: awk 'BEGIN {s=0} {s += \$(NF-1)} END {print s}' a.txt

27. How can you redirect on the command line the standard error through pipe to another program?

Answer: (std error is 2)

command 2>&1 | other_command

28. Write a UNIX Shell script that displays all command line arguments without using FOR

Answer: (we use while)

c=1

while [\$c -le \$#]

do

eval echo \\$\$c

c=`expr \$c + 1`

done

or we can use shift

while [\$c -le \$#]

do

echo \$1

shift

c=`expr \$c + 1`

done

29. Draw the hierarchy of processes created by the code below, including the parent process.

```

for(i=0; i<3; i++) {
    fork();
    execlp("ls", "ls", "/", NULL);
}

```

Answer: P -> C1

The parent forks and makes child C1 and then they both go on and execute the exec. But exec, by definition, ends the current process, so both P and C1 are done.

30. Add the necessary lines of C code so that the instruction below overwrites the content of a file b.txt. Do not change the instruction.

```

stdin -> 0
stdout -> 1
stderr -> 2
a.txt -> 3
b.txt -> 4

#include <unistd.h>

dup2(4, 1);
execlp("cat", "cat", "a.txt", NULL);

```

31. Why is it not advisable to communicate bidirectionally through a single FIFO?

Answer: Using one FIFO in both directions can cause data interleaving, ambiguity of message origin, and possible deadlocks. Instead, two FIFOs should be used, one for each direction. We wouldn't know who read and who wrote. Might get data corrupted.

32. How many FIFOs can a process open if they are not and will not ever be used by any other process?

Answer: 0, because the program crashes if we open a fifo and we don't have another process to open it for the complementary operation.

33. When would you prefer using a process instead of a thread?

Answer: when I have data/resources that I want to keep private, not have them shared with other processes. Also, since processes do not share memory(they are completely isolated), a failure in one process does not affect the other processes.

34. What is a "critical section"?

Answer: A critical section is a part of a program where shared resources are accessed and that must be executed by only one thread or process at a time to prevent race conditions. Let's say a simple instruction like `n++`, which modifies the global variable `n`.

35. Why should the thread recheck the condition after returning from the `pthread_cond_wait` call?

Answer: The condition must be rechecked after `pthread_cond_wait()` because wakeups are not guaranteed to correspond to the condition being satisfied due to spurious wakeups and race conditions. After the thread was signaled, in the meantime, maybe a thread already modified the resource, so the condition must be checked again.

36. What will be the effect of replacing calls to `pthread_mutex_lock` with calls to `pthread_rwlock_rdlock`?

Answer: the read-lock is a shared lock that can be obtained any number of times, as long as there is no write-lock present. In this case, all the threads are able to read the data (unlike when using a mutex, where only one thread is able to access the resource), but NOT write. However, if write operations exist and are not protected by write locks, this change can introduce race conditions. It's like shoppers in a shop, where some people buy, but other people only look at the product. In this case, we can have multiple people just looking at the product.

37. What is the effect of calling `pthread_barrier_wait` on a barrier initialized with 1?

Answer: If a barrier is initialized with value 1, calling `pthread_barrier_wait()` has no blocking effect. The calling thread immediately passes the barrier because the required number of threads (1) is already satisfied. Therefore, the barrier becomes effectively useless, as it does not enforce any synchronization.

38. How can you increment the value of a POSIX semaphore?

Answer: `sem_post(&sem)`

39. What can you do as a software developer to prevent deadlocks? Justify your answer.

Answer: Deadlocks happen because of 4 reasons: mutual exclusion (which we need), lock and wait (also needed), non-preemption and circular wait. The only one we can control per se is the circular wait one. To ensure that we won't create cycles (waiting on threads that are also waiting on someone else) always lock and unlock your resources in the same order.

40. What state transition will a process undergo when reading from a file?

Answer: from running, it goes to blocked (where it waits for data to be read), then back into ready, and when the CPU allows it, back to running

41. What is the content of the superblock on a Linux disk?

42. Enumerate the data block addressing types of an i-node.

Answer: direct pointers + simple indirect pointers + double indirect pointers + triple indirect pointers

43. Give three regular expressions that match any line that contains a least two vowels but no digits.

Answer:

```
“^[^0-9]*[aeiouAEIOU][^0-9]*[aeiouAEIOU][^0-9]*$”
```

```
“^([ ^0-9]*[aeiouAEIOU][^0-9]*){2}$”
```

```
“^[^0-9]*([aeiouAEIOU][^0-9]*){2,}$”
```

44. Give a GREP command that display all the lines in a file that contain an even number of vowels (among other potential characters).

Answer:

```
grep -E -i “^(^[aeiou]*[aeiou][^aeiou]*[aeiou][^aeiou]*)*$” file.txt
```

45. Write a SED command that display from a file only the lines that contain exclusively a valid arithmetical expression of integer additions and subtractions.

Answer:

```
sed -E -n “/^[^?][0-9]+([+][0-9]+)*/p” file.txt
```

46. Write an AWK command that displays the sum of the fields on the position equal to the current line number.

Answer:

```
awk 'BEGIN {sum=0} {if (NF>=NR) sum += $NR } END {print sum}' file.txt
```

47. Give two solutions for hiding a commands standard and error outputs by redirecting them to /dev/null.

Answer:

```
command > /dev/null 2>&1
```

```
command > /dev/null 2> /dev/null
```

48. Write a UNIX Shell script that asks the user for a directory name and insists until it gets a directory that does not exist already.

```
read dir
while [ -d $dir ]
do
    echo "this is already a directory"
```

```
read dir
done
```

49. Draw the hierarchy of processes created by the code below, including the parent process.

```
for(i=0; i<2; i++) {
    fork();
    execl("/etc", "/etc", NULL);
}
```

P->C1 (we've had it before as well)

50. Add the necessary lines of C code so that the instruction below overwrites the content of a file b.txt. Do not change the instruction.

```
#include <unistd.h>
#include <fcntl.h>

int fd = open("b.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, 1);
close(fd);

execlp("sort", "sort", "a.txt", NULL);
```

51. Why is it not advisable to communicate bidirectionally through a single pipe?

Answer: A single pipe is not suitable for bidirectional communication because pipes only support unidirectional byte streams. Using one pipe in both directions can cause data interleaving, ambiguity of message origin, and potential deadlocks. Therefore, two separate pipes should be used, one for each direction of communication.

(sidenote: it's like for fifo; fifos are after all just named pipes)

52. What will display the sequence of commands below, considering the f is created successfully?

```
mkfifo f
echo asdf > f
cat f
```

Answer: asdf?? (at a theoretical level, it should work, because we transmit the message with echo, and then we have a reader – cat. But in practice, if we run it on the command line, we need to add & after the echo)

53. What is a "critical resource"?

Answer: A critical resource is any shared resource (such as a variable, file) that must be accessed in a mutually exclusive manner because concurrent access by multiple processes or threads can lead to inconsistencies or race conditions.

54. Why does the `pthread_cond_wait` call get also a mutex as argument?

Answer: The mutex is passed to `pthread_cond_wait()` so that the function can atomically release the mutex and block the thread, preventing race conditions between checking the condition and going to sleep. Before returning, the mutex is re-acquired to ensure safe access to the shared state.

55. What will be the effect of replacing calls to `pthread_mutex_lock` with calls to `pthread_rwlock_wrlock`?

Answer: (we had a similar question) Replacing `pthread_mutex_lock()` with `pthread_rwlock_wrlock()` enforces exclusive access in the same way as a mutex, since a write lock prevents both readers and writers from entering the critical section. However, it introduces additional overhead because read-write locks are more complex and are intended to allow concurrent readers, which is not utilized in this case.
(when we have a write lock, we cannot have readers!!)

56. What is the effect of calling `sem_wait` on a semaphore with value zero?

Answer: If `sem_wait()` is called on a semaphore with value 0, the calling thread is blocked and placed in the semaphore's waiting queue. It remains blocked until another thread increments the semaphore using `sem_post()`, which may wake up one of the waiting threads and allow it to continue execution.

57. How can you decrement the value of a POSIX semaphore?

Answer: The value of a POSIX semaphore is decremented using `sem_wait()`, which decreases the semaphore value by 1 if it is greater than zero, or blocks the calling thread if the value is zero until it can be decremented.

58. Considering that a block can contain N addresses towards other blocks, how many data blocks are addressed by an i-node's double and triple indirections together?

59. Give a regular expression that matches any even-length sequence of lower-case words separated by spaces, if for each word its length and its position in the sequence are either both odd or both even. The words were counted starting with 1. Ex: the 5th word must have odd length and the 16th word must of even length.

Answer: `^([a-z]{2}*[a-z] [a-z]{2}+ ){0,}([a-z]{2}*[a-z] [a-z]{2}+)*$`
(no idea if it works ☹)

60. Give three GREP commands that display the lines of a file which consist exclusively of a non-empty sequence of alternating letters and digits (eg `a0g` or `1r5m`)

Answer:
`grep -E -i "^(^([0-9]?([a-z][0-9])*)$|^([a-z]?([0-9][a-z])*)$)"`

```
grep -E "^(^([0-9]?([a-zA-Z][0-9])*)$|^([a-zA-Z]?([0-9][a-zA-Z])*)$)"
```

61. Write two SED commands that display a file's lines deleting the first non-empty sequence of lower-case letters

Answer:

```
sed -E "s/[a-z]+//" file.txt
```

```
sed "s/[a-z]\+//" file.txt
```

62. Write an AWK command that displays the sum of all the numbers in a text file whose lines consist of sequences of digits separated by spaces.

Answer:

```
awk 'BEGIN {sum=0} {for (i=1;i<=NF;i++) sum += $i } END {print sum} '
```

63. Give three ways of finding the size of a file on the Linux command line.

Answer:

```
ls -lh file
```

```
du -h file
```

```
wc -c file
```

64. Write a UNIX Shell condition that checks whether a file exists and the current user has any permissions on it.

Answer:

```
//or this: if [ -f "$filename" ] && { [ -r "$filename" ] || [ -w "$filename" ] || [ -x "$filename" ]; }
```

```
if [ -f "$filename" ] && [ -r "$filename" -o -w "$filename" -o -x "$filename" ]
```

```
then
```

```
    echo "yes"
```

```
else
```

```
    echo "no"
```

```
fi
```

65. Draw the hierarchy of processes created by the code below, including the parent process.

```
for(i=0; i<3; i++) {  
    if(fork() > 0) {  
        wait(0);  
        wait(0);  
        exit(0);  
    }  
}
```

Answer:

P

☐ C1

☐ C2

☐ C3

66. What will the fragment below print? Justify your answer.

```
execl("expr", "expr", "1", "+", "1", NULL);
execlp("echo", "echo", "3", NULL);
printf("4\n");
```

Answer: 3 ; the first exec fails because we do not have the path given as argument. So it goes to the second exec and all instructions that follow are not considered

67. Explain why the file descriptor returned by popen must be closed with pclose instead of fclose.

Answer: The file descriptor returned by popen() must be closed using pclose() because it is associated with a pipe connected to a child process executing a shell command. pclose() not only closes the stream but also waits for the child process to terminate and collects its exit status, whereas fclose() only closes the stream and does not handle the associated process.
(on short: fclose() does not handle the process part)

68. How many FIFOs can a process open for reading if the FIFOs are and will ever be used by other processes only for writing?

Answer: A process can open as many FIFOs for reading as allowed by the system's limit on open file descriptors. Since FIFOs are represented as file descriptors, the only constraint is the per-process file descriptor limit, not the FIFO mechanism itself.

69. When would you use execv instead of execl?

Answer: execv is used instead of execl when the arguments are already stored in an array (vector form), such as an argv-style structure. execl requires passing arguments individually, while execv is more suitable for dynamically constructed argument lists.

70. Give three function calls that ensure mutual exclusion.

Answer:

Pthread_mutex_lock(&m);

Sem_wait(&sem);

Pthread_rwlock_rdlock(&rwlock);

71. What will be the effect of replacing calls to `pthread_mutex_lock` with calls to `sem_post`?
Answer: `sem_post()` allows any thread to go execute, but now the critical resources are not protected by the mutex anymore, which can lead to race condition and data corruption

72. What could happen if function `f` were executed by several simultaneous threads? Why?

```
pthread_mutex_t m[2];
void* f(void* p) {
    int id = (int)p;
    pthread_mutex_t* first = &m[id % 2];
    pthread_mutex_t* second = &m[(id+1) % 2];

    pthread_mutex_lock(first);
    pthread_mutex_lock(second);
    ...
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}
```

Answer: if one has the even id, and the other an odd id, then we might have a deadlock situation (we should always lock and unlock resources in the same order).

73. Give an example of distinct values greater than 0 for `T`, `N1`, `N2`, and `N3` for which the program below finishes execution.

```
pthread_barrier_t b1, b2;

void* f1(void* a) {
    pthread_barrier_wait(&b1);
    return NULL;
}

void* f2(void* a) {
    pthread_barrier_wait(&b2);
    return NULL;
}

int main() {
    int i;
    pthread_t t[T][2];

    pthread_barrier_init(&b1, NULL, N1);
    pthread_barrier_init(&b2, NULL, N2);
```

```

    for(i=0; i<T; i++) {
        pthread_create(&t[i][0], NULL, f1, NULL);
        pthread_create(&t[i][1], NULL, f2, NULL);
    }
    for(i=0; i<T; i++) {
        pthread_join(t[i][0], NULL);
        pthread_join(t[i][1], NULL);
    }
    pthread_barrier_destroy(&b1);
    pthread_barrier_destroy(&b2);
    return NULL;
}

```

Answer: $T=N1=N2=N3=6$

74. What state transition will a process undergo when it calls `sem_wait` and under what conditions? Justify your answer.

Answer: if the value of the semaphore is 0, then the process gets blocked until the semaphore gets incremented; if the value is > 0 , it decrements the sem and continues running

75. Considering that the size of a block is B and the size of address is A, how many data blocks are addressed by the double indirect addressing of an i-node?

Answer: a block of memory has B/A pointers

Double indirect is: `inod -> block -> block -> data block`

So we have 2 block, then $(B/A)^2$ (they get multiplied; if we had had triple indirect, then it would've been to the power of 3)

76. When would you load into memory the pages of a process that is just starting?

Answer: Pages of a process are loaded into memory when they are first referenced (on demand), not at process creation time, unless the OS uses optional preloading techniques.

77. Scrieți un script Shell UNIX care afișează toate argumentele din linia de comandă fără a folosi FOR. EN: Write a UNIX Shell script that displays all command line arguments without using FOR

```
echo "$@"
```

78. Can you create a hard link towards a file on a different partition? Justify your answer.

Answer: no, they have to be on the same filesystem, because they reference an i-node and i-nodes are unique to their filesystem.

79. Give a regular expression that matches any odd number of words, each word having an odd number of letters.

Answer:

```
^(((([a-z][a-z]*[a-z])) ([a-z][a-z]*[a-z])))* ((([a-z][a-z]*[a-z]))$
```

80. Give four commands that display the number of empty lines in a file.

Answer:

```
grep -E -c “^$” file
```

```
grep -E -c -v “^.$” file
```

```
sed -n -E “/^$/p” file | wc -l
```

```
awk ‘BEGIN {count=0} /^$/ {count++} END {print count}’ file
```

81. Write a SED command that displays a file's lines deleting the first, the third, the fifth, the seventh, etc space on those lines.

Answer: `sed -E “/([ ^ ]*)( )([ ^ ]*)( )\1\3\4/gi” a.txt`

82. Write an AWK command that displays the product of the last field of lines on odd positions having an odd number of fields.

Answer:

```
awk ‘BEGIN {p=1} {if (NR%2=1 && NF%2=1) p*= $NF} END {print p}’ a.txt
```

83. Give four ways of redirecting the standard output of a process.

Answer:

```
Process > a.txt
```

```
Process >> a.txt
```

```
Process | tee file
```

```
Process 1> a.txt
```

84. Write three UNIX Shell conditions that check the existence of a file.

```
[ -f “$filename” ]
```

```
[[ -f “$filename” ]]
```

```
test -f “$filename”
```

85. Draw the hierarchy of processes created by the code below, including the parent process.

```
for(i=0; i<3; i++) {  
    if(execlp("ls", "ls", "/", NULL) != -1) {  
        fork();  
    }  
}
```

Answer: we only have P

We execute the exec and P terminates

86. Add the necessary code so that the instruction below does not get stuck waiting for standard input.

```

    execlp("cat", NULL);

#include <unistd.h>
#include <fcntl.h>
Int fd=open("a.txt", O_RDONLY);
Dup2(fd,0);
Close(fd);

```

87. When would you prefer using a FIFO instead of a pipe?

Answer: we do not need a common ancestor, they live longer than the process (this also means that they need to be manually deleted)

88. When would you prefer using a mutex instead of a rwlock?

Answer: when we wish to have exclusive possession of a resource. Mutex allows only one process/thread to read/write data, while for rwlock we might have multiple readers. And mutexes are also less prone to deadlocks.

89. What will be the effect of replacing calls to `pthread_mutex_lock` with calls to `sem_wait`?

Answer: the semaphore might allow a thread to pass, even if there's another thread in execution which might be using a critical resource. And this can lead to race condition and data corruption.

90. What does `pthread_cond_wait` do with the mutex it gets as argument?

Answer: it unlocks the mutex, it blocks the thread (the thread goes to sleep while waiting for the waking signal). When the thread wakes up it acquires the mutex, ensuring safe access to the shared data and preventing race conditions

91. What state transition will a process undergo when it calls `pthread_cond_wait`? Justify your answer.

Answer: it gets blocked, then it waits for the waking up signal; when it receives it, it becomes ready and gets back to running state

92. What is the content of file of type directory in the Linux file system?

Git answer: -directory entries(file names, inode numbers)
 -.(dot) represent the directory, has the inode of the directory
 -..(dot dot) represent parent directory, inode of parent
 -regular entries, name file or subdirectory and inode specific

93. Explain the difference between a symbolic link and a hard link.

Answer: a hard link references and i-node in the filesystem. While a symbolic link references another file

94. Give a regular expression that accepts strings containing only the sequence "abc", repeated at least 3 times, regardless whether it is upper or lower case.

Answer: ([aA][bB][cC]){3,}

95. Write a GREP command that displays the lines of odd length that start and end with a vowel.

Answer: grep -E -i “^[aeiou].(..)*[aeiou]\$” a.txt

96. Consider a text file that contains at least 4 words (non-empty sequences of uppercase or lowercase letters) separated by SPACES on each line. Write two SED commands that swap the second word with the next-to-last word on each line of this file.

Answer: sed -E “s/^[a-z]+)([a-z]+)([a-z]+)*([a-z])([a-z])\$\1\4\3\2\5/i”

97. Write an AWK command that displays the product of the second and next-to-last field of each line if the number of fields on the line is at least 3.

Answer: awk ‘{if (NF>=3) { p=\$2* \$(NF-1); print p}}’ a.txt

98. Write a command line in which program B is executed only if program A has executed successfully.

Answer: mkdir project_nou && cd project_nou

99. Write a Shell script that reads from file a.txt pairs of file names and deletes the second file if the first already exists.

Answer:

```
while read -r fisier1 fisier2
do
    if [ -f $fisier1 ]
    then
        rm -f $fisier2
    fi
done
```

100. Considering that the fork call is executed successfully, how many child processes will be created by the program below? Justify your answer.

```
int main() {  
    while(wait(NULL) > 0)  
        if(fork() == 0) {  
            exit(0);  
        }  
    return 0;  
}
```

Answer: the `wait(NULL)` evaluates to 0 because the process does not have any children yet. So the fork is not executed. Thus we have 0 child processes.